

Средства за реализация

Създаването на пространствено разположени криви и равнини в OpenGL е осъществено чрез криви на Безие и базисни сплайни (NURBS) чрез елементите на библиотеките Gl и Glu.

За създаване и визуализация на **криви на Безие** с Gl библиотека се използват два варианта:

Вариант 1 – Последователност на работа :

- I. Дефиниране на кривата с *glMap1* чрез масив от координати на точки**
- II. Вмъкване с *glEnable***
- III. Задаване на оцветяване, режим на оцветяване и свързване на точките от кривата**
- IV. Извикване и изчертаване чрез вмъкване на *glEvalCoord1* между *glBegin* и *glEnd***

Задаване на параметрите на кривата. Използват се за описание на крива, цвят и текстура.

glMap1f/d (GLenum target, TYPE t1, TYPE t2, GLint stride, GLint order, const TYPE *points)

target – показва типа на контролните точки:

формат на върхове – GL_MAP1_VERTEX_3/4 – 3 или 4 стойности, съответстващи на x,y,z и w

формат на цвета – GL_MAP1_INDEX – индексен цвят или
GL_MAP1_COLOR_4 – RGBA

нормали – GL_MAP1_NORMAL – x,y,z за координати на нормала към точка

текстурни координати - GL_MAP1_TEXTURE_COORD_1/2/3/4 – 1/2/3 или 4 стойности на координата на текстура

t1,t2 – обхват на изменение на параметъра t (между 0 и 1)

tstride – брой нива между контролните точки – за четворка точки стойността на параметъра е 3

order – степен на полинома +1= брой контролни точки ***points** – указател към масива с координатите на контролните точки

Пресмятане на стойностите на параметризиращата променлива –t

glEvalCoord1f/d/v (GLfloat t)

Вариант 2 - Последователност на работа:

- I. Дефиниране на кривата с *glMap1* чрез масив от координати на точки**
- II. Вмъкване с *glEnable***
- III. Задаване на оцветяване, режим на оцветяване и свързване на точките от кривата**
- IV. Извикване и изчертаване чрез създаване на решетка – *glMapGrid1* и пресмятане на координатите по нея с *glEvalMesh1***

glMapGrid1f(GLint tn, GLfloat t1, GLfloat t2);

tn – брой интервали в решетката

t1,t2 – стойности на параметъра за началния и крайния индекс в решетката

glEvalMesh1(GLenum mode, GLint i1, GLint i2);

mode – начин на визуализация GL_POINT – точки

или GL_LINE – линия

i1, i2 – начален и краен индекс за решетката

За създаване и визуализация на криви чрез **базисни сплайни** се използват функции от Glu библиотека. Последователността на работа изисква:

- I. Задаване на масиви за контролни точки и вектори на възли**
- II. Дефиниране на NURBS обект с *gluNewNurbsRenderer* и задаване на параметри за визуализация чрез *gluNurbsProperty***
- III. Визуализация на кривата с *gluNurbsCurve* между *gluBeginCurve* и *gluEndCurve***

Ако направленията са **u** и **v**, масивът от контролни точки има формат **array[u][v][3]** (**array[u][v][0]** задава **x**, **array[u][v][1]** задава **y**, **array[u][v][2]** задава **z**).

Крива се стартира чрез дефиниране на обект и задаване на параметри за него чрез:

gluNewNurbsRenderer(void) – връща указател към нов NURBS обект

gluDeleteNurbsRenderer(GLUnurbs *nobj) – изтрива обект ***nobj**,

генериран с горната функция

gluNurbsProperty (GLUnurbs

***nobj, GLenum property, GLfloat value)** – задава параметри на обекта

***nobj**

property – указва характеристика, която получава стойност от следващия параметър във функцията. Могат да бъдат:

GLU_SAMPLING_TOLERANCE – задава максимална дължина в пиксели, използвана, когато методът на вземане на отчети (смплиране) е **GLU_PATH_LENGTH**. По подразбиране стойността е 50.0 пиксела.

GLU_DISPLAY_MODE – стойността на параметъра задава как се рендира NURBS повърхнина. Следващият параметър **value** може да бъде **GLU_FILL** – запълнени полигони, **GLU_OUTLINE_POLYGON** – линии на полигоните или **GLU_OUTLINE_PATCH** – линии на допълнена или изрязана част,

GLU_CULLING – **value** приема стойност истина **GL_TRUE** – означава изрязване на пикселите извън текущия изглед; по подразбиране стойността е лъжа **GL_FALSE**,

GLU_AUTO_LOAD_MATRIX – **value** приема стойност истина **GL_TRUE** или **GL_FALSE**.

GLU_PARAMETRIC_TOLERANCE – задава максимална дистанция в пиксели, при метод на смплиране **GLU_PARAMETRIC_ERROR**. По подразбиране стойността е 0.5,

GLU_SAMPLING_METHOD – указва как се съставя мозайката при повърхнина; **value** може да приема параметри: **GLU_PATH_LENGTH**, **GLU_PARAMETRIC_ERROR** или **GLU_DOMAIN_DISTANCE**,

GLU_U_STEP – задава броя точки на единица дължина по *u* направление Използва се когато **GLU_SAMPLING_METHOD** е зададен като **GLU_DOMAIN_DISTANCE**. По подразбиране стойността на **value** е 100.

GLU_V_STEP – задава броя точки на единица дължина по *v* направление Използва се когато **GLU_SAMPLING_METHOD** е зададен като **GLU_DOMAIN_DISTANCE**. По подразбиране стойността на **value** е 100.

value – в зависимост от предходния параметър може да приема числена стойност или параметър:

GLU_PATH_LENGTH – по подразбиране. Указва, че повърхнината, рендирана с максимална дължина на полигоните в мозайката (в пиксели), не е по-голяма от стойността, зададена в **GLU_SAMPLING_TOLERANCE**

GLU_PARAMETRIC_ERROR – указва, че при рендиране на повърхнината стойността на **GLU_PARAMETRIC_TOLERANCE** задава

максимална дистанция (в пиксели) между полигоните на мозайката и апроксимираната повърхнина,

GLU_DOMAIN_DISTANCE – задава в параметрични координати колко точки на единица дължина има в направление *u* или *v*.

gluBeginCurve(GLUnurbs *nobj) – стартира кривата, дефинирана като обект

gluNurbsCurve(GLUnurbs *nobj, GLint nknots, GLfloat *knot, GLint stride, GLfloat *ctlarray, GLint order, GLenum type) – описание на параметрите на изчертаваната крива

***nobj** – генериран обект

nknots – брой възли за параметъра

***knot** – масив с ненамаляващи стойности за вектори на възлите

stride – брой нива (брой стойности) в масива между контролните точки

***ctlarray** – масив от контролни точки

order – ред на сплайна = степен на полинома+1

type – тип на кривата, може да бъде GL_MAP1_VERTEX_3 или GL_MAP1_COLOR_4, или GL_MAP1_NORMAL

gluEndCurve(GLUnurbs *nobj) – приключва кривата

Съставни криви от линейни сегменти могат да бъдат създавани чрез запазване на предишните координати с **glPushAttrib(GLU_EVAL_BIT)** и възстановяването им с **glPopAttrib()**.

Обслужване на повърхнини

За създаване и визуализация на повърхнини чрез **криви на Безие** в GL библиотека се използват аналогични за кривите два варианта:

Вариант 1 - Последователност на работа :

- I. Дефиниране на кривата с *glMap2* чрез масив от координати на точки**
- II. Вмъкване с *glEnable***
- III. Задаване на оцветяване, режим на оцветяване и свързване на точките от кривата**
- IV. Извикване и изчертаване чрез вмъкване на *glEvalCoord2* между *glBegin* и *glEnd***

Задаване на параметрите на кривата:

glMap2f/d (GLenum target, TYPE t1, TYPE t2, GLint tstride, GLint torder, TYPE s1, TYPE s2, GLint sstride, GLint sorder, const TYPE *points)

target – показва типа на контролните точки:

формат на върхове – GL_MAP1_VERTEX_3/4 – 3 или 4 стойности съответстващи на x,y,z и w

формат на цвета – GL_MAP1_INDEX – индексен цвят или GL_MAP1_COLOR_4 – RGBA

нормали – GL_MAP1_NORMAL – x,y,z за координати на нормала към точка

текстурни координати – GL_MAP1_TEXTURE_COORD_1/2/3/4 – 1/2/3 или 4 координати на текстурата

t1,t2 – обхват на изменение на първия параметър

tstride – брой нива (брой стойности) в масива между контролните точки за първия параметър

torder – степен на полинома +1

s1,s2 – обхват на изменение на втория параметър

sstride – брой нива (брой стойности) в масива между контролните точки за втория параметър

sorder – степен на полинома +1

***points** – указател към масива с координатите на контролните точки

Пресмятане на стойностите на параметризиращите променливи – **t** и **s** чрез

glEvalCoord2f/d/v (GLfloat t, GLfloat s)

Вариант 2 – Последователност на работа :

- I. Дефиниране на кривата с **glMap2** чрез масив от координати на точки
- II. Вмъкване с **glEnable**
- III. Задаване на оцветяване, режим на оцветяване и свързване на точките от кривата
- IV. Извикване и изчертаване чрез създаване на решетка – **glMapGrid2** и пресмятане на координатите по нея с **glEvalMesh2**

glMapGrid2f (GLint tn, GLfloat t1, GLfloat t2, GLint sn, GLfloat s1, GLfloat s2);

tn – брой интервали в решетката в направлението на **t**

t1,t2 – стойности на параметъра за началния и крайния индекс в решетката

sn – брой интервали в решетката в направление на **s**
s1,s2 – стойности на параметъра за началния и крайния индекс в решетката

glEvalMesh2(GLenum mode, GLint i1, GLint i2, GLint j1, GLint j2);

mode – начин на визуализация **GL_POINT** – точки или **GL_LINE** – линия

i1, i2 – начален и краен индекс за решетката по направление на **t**

j1, j2 – начален и краен индекс за решетката по направление на **s**

Създаването и визуализацията на повърхнини чрез **базисни сплайни** ползва същата последователност както при криви. Контролните точки се определят с координати по двете направления на повърхнината, определени в тримерното пространство. Ако направленията са **u** и **v**, масивът от контролни точки има формат **array[u][v][3]**

(**array[u][v][0]** задава **x**, **array[u][v][1]** задава **y**, **array[u][v][2]** задава **z**). Броят точки, задавани във всяко направление трябва да бъде допълнен с брой точки, определен от реда на сплайна – **m**, по условието $2*(m-1)$.

Използваната функция за изчертаване в случая е **gluNurbsSurface** поставена между **gluBeginSurface** и **gluEndSurface**.

gluBeginSurface(GLUnurbs *nobj) – стартира повърхнина, дефинирана като обект

gluEndSurface (GLUnurbs *nobj) – приключва повърхнината

gluNurbsSurface(GLUnurbs *nobj, GLint sknot_count, float *sknot, GLint tknot_count, GLfloat *tknot, GLint s_stride, GLint t_stride, GLfloat *ctlarray, GLint sorder, GLint torder, GLenum type);

***nobj** – генериран обект

sknot_count – брой възли за параметъра **s**

***sknot** – масив с ненамаляващи стойности на вектори на възлите за **s**

tknot_count – брой възли за параметъра **t**

***tknot** – масив с ненамаляващи стойности на вектори на възлите за **t**

s_stride, t_stride - брой нива (брой стойности) в масива между контролните точки във всяко направление

***ctlarray** – масив от контролни точки

order – ред на сплайна = степен на полинома+1
type – тип на кривата, може да бъде GL_MAP1_VERTEX_3 или GL_MAP1_COLOR_4, или GL_MAP1_NORMAL

За повърхнините, изпълнени с NURBS, е налична възможност за изрязване на части от повърхнината чрез задване на масив от точки, определящи границите на отделяната област под формата на затворена крива. За целта се използват функции:

gluBeginTrim(GLUnurbs *nobj) – стартира изрязването от повърхнина, дефинирана като обект

gluPwlCurve(GLUnurbs *nobj, GLint count, GLfloat *array, GLint stride, GLenum type); – ограничава линеен B-сплайн

***nobj** – генериран обект

count – брой точки от кривата

***array** – масив, съдържащ координатите на точките от кривата

stride – брой нива (брой стойности) в масива между контролните

точки

type – тип на кривата, в зависимост от описанието на кривата с x и y, или с x,y,w може да бъде GL_MAP1_TRIM_2 или GL_MAP1_TRIM_3

gluEndTrim(GLUnurbs *nobj) – приключва изрязването.

Glu библиотеката предоставя и възможности за пряко изчертаване и манипулиране на повърхнини от втора степен.

gluNewQuadric – генерира обект с описание от втора степен.

gluDeleteQuadric – изтрива генериран квадратичен обект.

gluCylinder(GLUquadric

***qobj, GLdouble baseRadius, GLdouble topRadius,**

GLdouble height, GLint slices, GLint stacks); – създава и визуализира цилиндрична повърхност

***qobj** – генериран обект

baseRadius – радиус при основата

topRadius – радиус при височината

height – височина на цилиндъра

slices, stacks – паралели и меридиани по z

gluDisk(GLUquadric

***qobj, GLdouble innerRadius, GLdouble outerRadius, GLint slices, GLint loops);**

***qobj** – генериран обект

innerRadius – вътрешен радиус

outerRadius – външен радиус

slices – брой окръжности около z оста

loops – брой концентрични кръгове на които е разделен дискът

gluSphere(GLUquadric *qobj, GLdouble radius, GLint slices, GLint stacks
);

***qobj** – генериран обект

radius – радиус

slices, stacks – паралели и меридиани по z